The n-Queens Problem

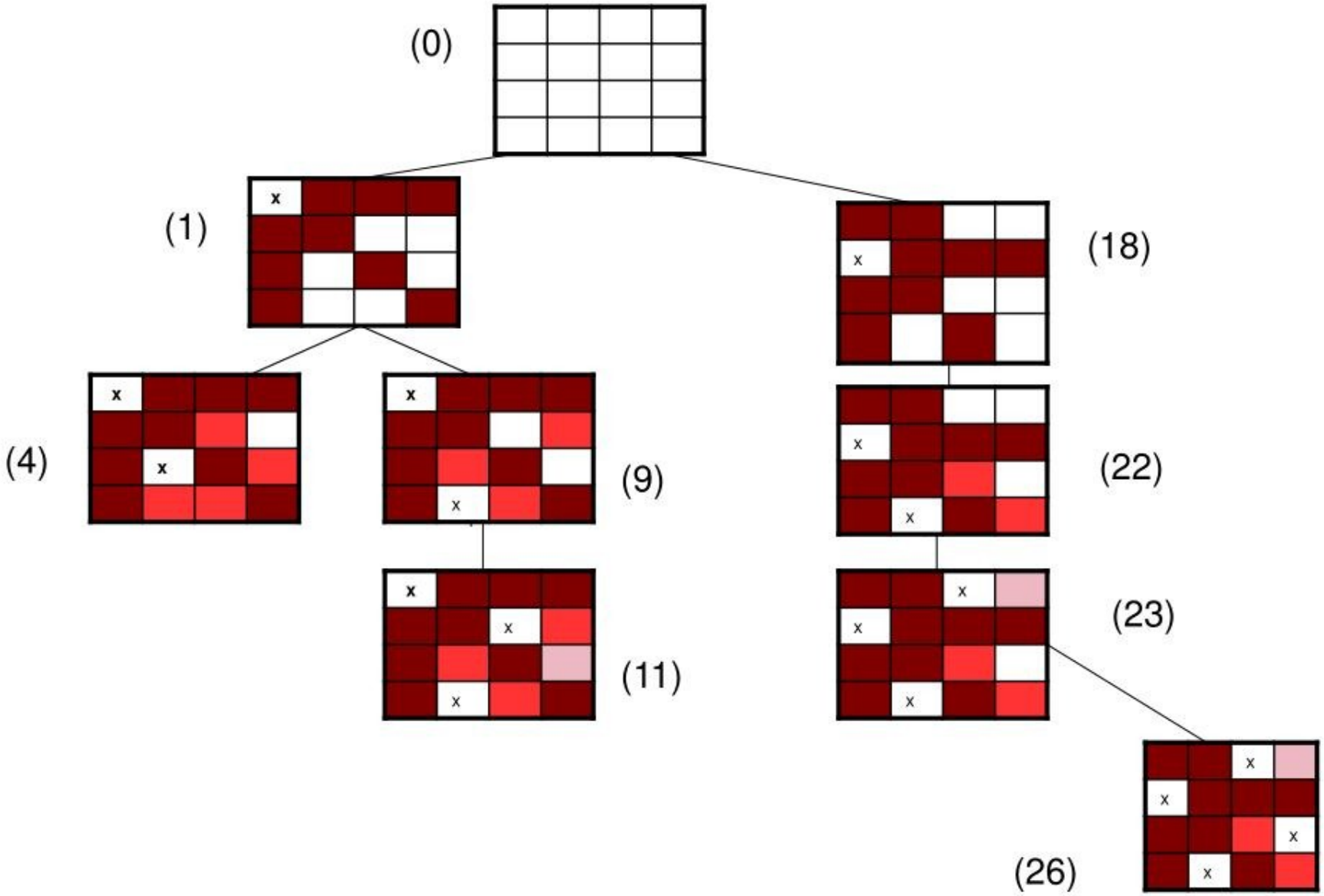
- The problem is to place n queens (chess pieces) on an n by n board so that no two queens are in the same row, column or diagonal

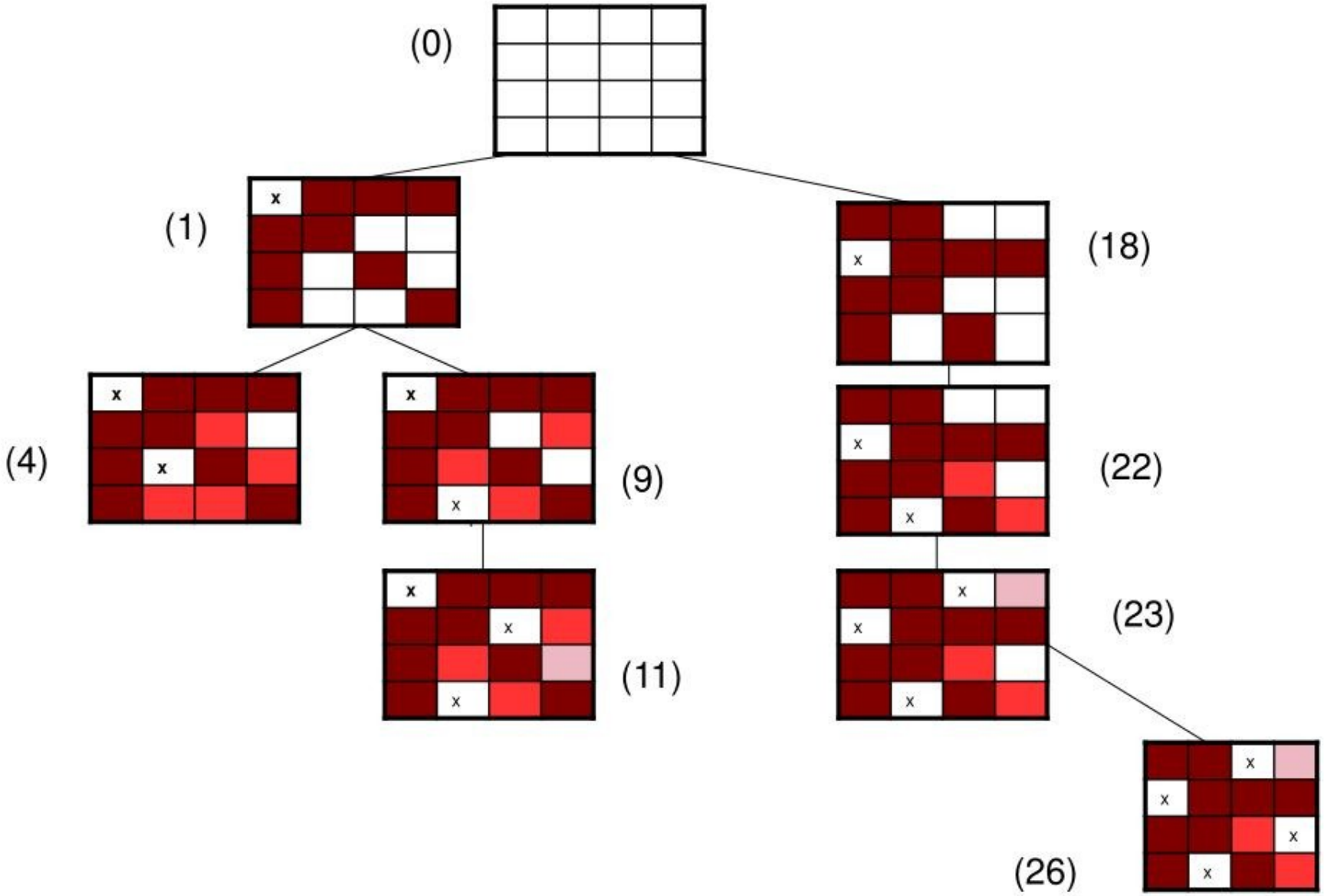
		Q	
Q			
			Q
	Q		

- The 4 by 4 board above shows a solution for $n = 4$
- The text gives an explicit construction for a solution for each $n \geq 4$ (page 202)
- But first we will introduce an algorithm strategy called ***backtracking***, which can be used to construct ***all*** solutions for a given n.

The n-Queens Problem

- We can solve this problem by generating a dynamic tree in a depth-first manner
- We will try to place a queen in each column from the first to the last
- For each column, we must select a row
- We start by placing a queen in row 1 of the first column
- Then we check the rows in the second column for positions that do not conflict with our choice for the first column
- After placing a queen in the second column, we continue to the third, etc.
- If at any point we find we cannot place a queen in the current column, we back up to the previous column and try a different row for that column
- We then continue trying to place more queens
- This process can be visualized by a tree of configurations
- The nodes are partial solutions of the problem
- The root is an empty board
- The children of the root will be boards with placements in the first column
- We will generate the nodes of the tree dynamically in a depth-first way


(0)
(1)
(18)
(4)
(9)
(22)
(11)
(23)
(26)


(0)
(1)
(18)
(4)
(9)
(22)
(11)
(23)
(26)

Backtracking Solution to n-Queens Problem

- We could continue with the previous example to obtain all possible solutions for n = 4
- Our goal now is to convert this approach into an explicit algorithm
- We track the position of the queens by using an array row
- row[k] is the row in column k containing a queen
- The key function is the recursive function rn_queens
- When rn_queens(k,n) is called, queens have been successfully placed in columns 1 to k-1
- The function then attempts to place a queen in column k
- If it is successful, then
  - if k == n, it prints the solution
  - if k < n, it makes the call rn_queens(k+1,n)
  - it then returns to the call rn_queens(k-1,n)

Backtracking Solution to n-Queens Problem

- To test for a valid position for a queen in column k, we use a function ***position_ok(k,n)*** which returns true if and only if the queen tentatively placed in column k does not conflict with the queens in positions 1 to k-1
- The queens in columns i and k conflict if
  - they are in the same row: row[i] == row[k]
  - or in the same diagonal: | row[k] – row[i] | == k – i
- We are thus led to our first version of the functions

Backtracking Solution to n-Queens Problem

```
n_queens(n) {
  rn_queens(1,n)
}

rn_queens(k,n) {
   for row[k] = 1 to n
      if (position_ok(k,n) )
         if ( k == n ) {
            for i = 1 to n)
                 print(row[k] + "  ")
            println
         }
         else
             rn_queens(k+1,n)
}

position_ok(k,n) {
  for i = 1 to k-1
      if (row[k] == row[i]  ||  abs(row[k]-row[i]) == k-i)
                return false
  return true
}
```

Running Time

- Improvements can be made so that position_ok(k,n) runs in $O(1)$ time rather than $\boldsymbol{O}(k)$ time. See the text.
- We will will obtain an upper bound for the running time of our algorithm by bounding the number of times rn_queens(k,n) is called for each k<n
  - **k = 1**: **1** time, by n_queens
  - **1 < k < n**: at most **n(n-1)···(n-k+2)**, since there are n(n-1)···(n-k+2) ways to place queens in the first k-1 columns in distinct rows
- Ignoring recursive calls, rn_queens(k,n) executes in $\Theta(n)$ time for k < n.
- This gives a worst-case bound for rn_queens(k,n) of

  n [n(n-1)···(n-k+2)] for 1 < k < n
- For k = n, there is at most one placement possible for the queen. Also, the loop in rn_queens executes in $\Theta(n)$ time.
- There are n(n-1)···2 ways for the queens to have been placed in the first n-1 columns, so the worst case time for rn_queens(n,n) is

  n [n(n-1)···2]

Running Time

- Combining the previous bounds, we get the bound

$$
\begin{aligned}
& n\,[\,1 + n + n(n-1) + \cdots + n(n-1)\cdots 2] \\
= \; & n \cdot n!\,[\,1/n! + 1/(n-1)! + \cdots + 1/1!\,]
\end{aligned}
$$

- A result from calculus: $e = \sum_{i=0}^{\infty} \frac{1}{i!} = 1 + \sum_{i=1}^{\infty} \frac{1}{i!}$

- This means that $n \cdot n!\,[\,1/n! + 1/(n-1)! + \cdots + 1/1!\,] \leq n \cdot n!\,(e-1)$

- We thus have that our algorithm runs in $O(n \cdot n!)$ **time**